EV Public Charging Session Billing and Wallet Engine

Project Information

- **Team ID:** E008
- **Problem ID:** ECE-025
- **Project Title:** EV Public Charging Session Billing and Wallet Engine
- **Team Members:** Deekshith S and Deepika P
- **Section:** ECM A
- **Programming Language:** Python 3.10+
- **Testing Framework:** Python unittest

---

1. Problem Statement

Electric vehicle (EV) public charging stations need an effective system to manage charging sessions, calculate bills, process wallet transactions, and generate revenue reports.

Manual billing and wallet management can lead to calculation errors, incorrect transactions, and difficulties in tracking charging activities.

This project develops a Python-based backend application that automates EV charging session billing, wallet management, transaction tracking, and daily revenue reporting.

2. Project Overview

The EV Public Charging Session Billing and Wallet Engine is a backend application developed using Python.

The system reads charger details, user information, charging sessions, and wallet top-ups from CSV files. It processes events in chronological order, calculates invoices according to predefined business rules, updates wallet balances, and generates reports.

The application uses command-driven processing and CSV files for input and output. No graphical user interface is required.

3. Project Objectives

1. Register and manage EV charging stations.
2. Register users and maintain their membership details.
3. Read charging session information from CSV files.

4. Calculate energy, time, and idle charges.
5. Apply membership discounts and GST.
6. Maintain user wallet balances and transaction history.
7. Block users whose wallet balance becomes negative.
8. Process wallet top-ups and applicable bonus amounts.
9. Generate daily revenue reports for each charger.
10. Generate a list of blocked users.
11. Validate the system using automated test cases.

4. Scope and System Boundaries

Scope

The system includes:

- Charger registration and management.
- User registration and wallet management.
- Charging session processing.
- Invoice generation.
- Membership discount calculation.
- GST calculation.
- Wallet debit and credit transactions.
- Top-up processing and bonus calculation.
- Blocked-user management.
- Daily charger-wise revenue reports.
- Automated testing.

System Boundaries

- The application is a backend-only Python project.
- CSV files are used for storing input data.
- Commands are provided through `commands.txt`.
- The system does not include a mobile application or web interface.
- The system does not directly connect to physical EV chargers or payment gateways.

5. Technology Stack

Technology	Purpose
Python 3.10+	Main programming language
CSV files	Input and output data storage
Visual Studio Code	Development environment
Python unittest	Automated testing

Technology	Purpose
GitHub	Source code and project documentation management

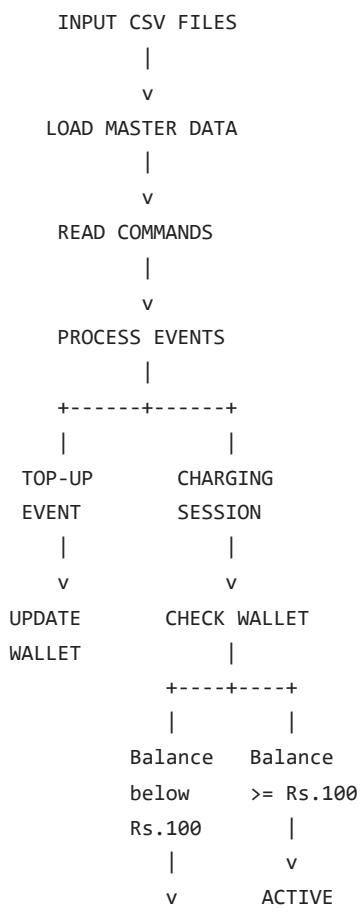
6. System Architecture and Workflow

The system follows a file-driven processing architecture.

System Workflow

1. Load charger details from `chargers.csv`.
2. Load user details from `users.csv`.
3. Load charging sessions from `sessions.csv`.
4. Load wallet top-ups from `topups.csv`.
5. Read commands from `commands.txt`.
6. Process events in chronological order.
7. Check wallet balance before starting a charging session.
8. Calculate the invoice when a session ends.
9. Update the wallet ledger and balance.
10. Generate daily revenue reports.
11. Generate the blocked-users report.

System Flow Diagram



```
REFUSED      |
             v
       CALCULATE BILL
             |
             v
       DEBIT WALLET
             |
             v
       UPDATE LEDGER
             |
             v
      GENERATE REPORTS
             |
             v
       OUTPUT CSV FILES
```

7. System Modules

7.1 Charger Management

Stores charger ID, charger type, location, and city.

7.2 User Management

Stores user ID, name, mobile number, membership tier, and opening wallet balance.

7.3 Session Processing

Reads charging session details and processes plug-in, charge-complete, and unplug events.

7.4 Invoice Calculation

Calculates energy charges, time charges, idle charges, discounts, GST, and final invoice amounts.

7.5 Wallet Management

Maintains wallet balances and records all wallet transactions in the ledger.

7.6 Top-up Management

Processes wallet top-ups, rejects invalid amounts, and adds bonus amounts for eligible top-ups.

7.7 Revenue Reporting

Generates daily revenue reports for each charger.

7.8 Automated Testing

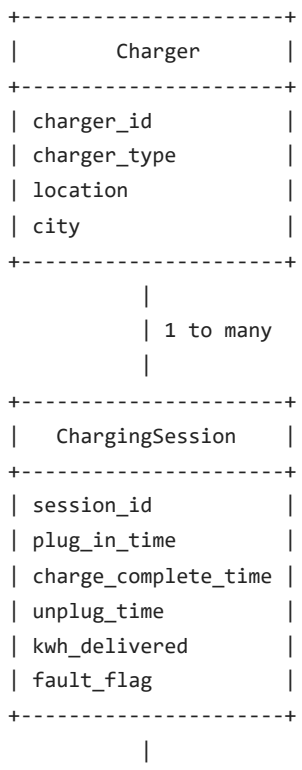
Uses Python unittest to verify billing calculations, wallet operations, and important edge cases.

8. Glossary

Term	Meaning
EV	Electric Vehicle
AC	Alternating Current
DC	Direct Current
kWh	Kilowatt-hour
GST	Goods and Services Tax
Invoice	Final charging bill
Wallet	User's available money balance
Ledger	Record of wallet transactions
Idle Time	Time between charging completion and unplugging
Top-up	Adding money to a user's wallet
Tariff	Price charged per unit of electricity

9. Conceptual Class Diagram

The following diagram represents the main entities and their relationships.



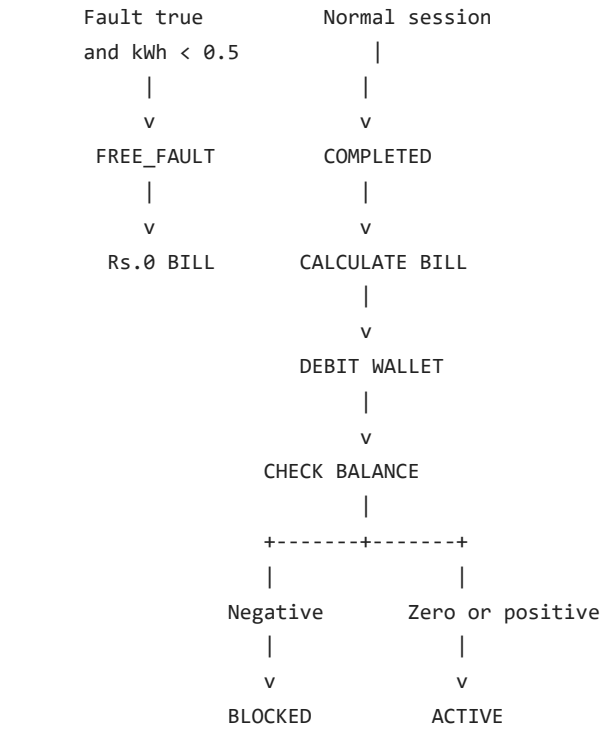
```
           | creates
           |
+----------------------+
|       Invoice        |
+----------------------+
| energy_rs            |
| time_rs              |
| idle_rs              |
| discount_rs          |
| gst_rs               |
| total_rs             |
+----------------------+

+----------------------+
|         User         |
+----------------------+
| user_id              |
| user_name            |
| tier                 |
| wallet_balance       |
+----------------------+
           |
           | has
           |
+----------------------+
|     WalletLedger     |
+----------------------+
| timestamp            |
| entry_type           |
| amount_rs            |
| balance_rs           |
+----------------------+

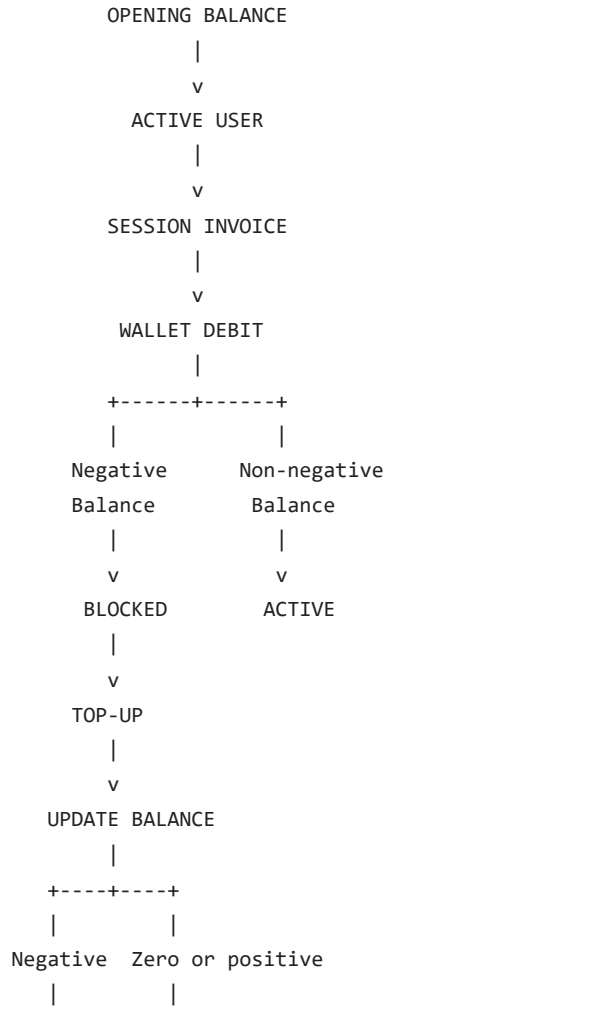
User also makes TopUps.
```

10. Session State Diagram

```
          NEW SESSION
               |
               v
          CHECK WALLET
               |
          +-----+-----+
          |           |
      Below Rs.100  At least Rs.100
          |           |
          v           v
       REFUSED      ACTIVE
                      |
                      v
                 SESSION END
                      |
             +--------+--------+
             |                 |
```



11. Wallet State Diagram



```
      v          v
  BLOCKED     ACTIVE
```

12. Business Rule Register

Rule ID	Business Rule
BR-01	AC_7KW tariff is Rs.12 per kWh.
BR-02	DC_30KW tariff is Rs.18 per kWh.
BR-03	DC_60KW tariff is Rs.22 per kWh.
BR-04	DC chargers have a time charge of Rs.1 per connected minute beyond 60 minutes.
BR-05	AC chargers do not have a time charge.
BR-06	The first 10 idle minutes are free.
BR-07	Idle time after the free period costs Rs.2 per minute.
BR-08	Idle charges are limited to Rs.300 per session.
BR-09	SILVER membership receives a 5% discount on energy charges only.
BR-10	GOLD membership receives a 10% discount on energy charges only.
BR-11	GST is 18% on energy, time, and idle charges after discount.
BR-12	The minimum final session charge after GST is Rs.20, except for free faulted sessions.
BR-13	A faulted session with energy below 0.5 kWh is free.
BR-14	A session can start only when the wallet balance is at least Rs.100.
BR-15	If an invoice exceeds the wallet balance, the wallet can become negative and the user is blocked.
BR-16	A blocked user remains blocked until a top-up brings the wallet balance to zero or above.
BR-17	Top-ups below Rs.100 are rejected. Top-ups of Rs.1,000 or more receive a 3% bonus.
BR-18	Wallet events are processed in chronological order, with top-ups first, session ends second, and session starts third when timestamps are equal.

13. Invoice Calculation

The invoice is calculated using the following components.

13.1 Energy Charge

Energy charge is calculated using the energy delivered and the charger tariff.

Formula:

Energy Charge = kWh Delivered × Tariff per kWh

13.2 Time Charge

Time charge applies only to DC chargers.

- First 60 connected minutes are free.
- Each additional connected minute costs Rs.1.

Formula:

Time Charge = Maximum of (0, Connected Minutes − 60) × Rs.1

13.3 Idle Charge

- First 10 minutes after charge completion are free.
- Each additional minute costs Rs.2.
- Maximum idle charge is Rs.300.

Formula:

Idle Charge = Minimum of (Maximum of (0, Idle Minutes − 10) × Rs.2, Rs.300)

13.4 Membership Discount

- SILVER: 5%
- GOLD: 10%
- Other tiers: 0%

The discount applies only to the energy charge.

13.5 GST

GST is 18% on the amount after applying the energy discount.

Formula:

Taxable Amount = Energy Charge + Time Charge + Idle Charge − Discount

GST = Taxable Amount × 18%

13.6 Final Invoice

Final Invoice = Taxable Amount + GST

The minimum final session charge after GST is Rs.20. This minimum does not apply to free faulted sessions.

13.7 Example Calculation

Consider a DC_30KW charging session with:

- Energy delivered: 10 kWh
- Connected time: 90 minutes
- Idle time: 15 minutes
- Membership: SILVER

Component	Calculation	Amount
Energy charge	10 × Rs.18	Rs.180.00
Time charge	(90 − 60) × Rs.1	Rs.30.00
Idle charge	(15 − 10) × Rs.2	Rs.10.00
Discount	5% of Rs.180	Rs.9.00
Taxable amount	180 + 30 + 10 − 9	Rs.211.00
GST	18% of Rs.211	Rs.37.98
Final invoice	211 + 37.98	**Rs.248.98**

14. Wallet and Top-up Management

The wallet module maintains the user's available balance and records every wallet transaction.

Wallet Operations

1. Opening balance is recorded for each user.
2. A minimum balance of Rs.100 is required to start a charging session.
3. The invoice amount is debited when the session ends.
4. If the balance becomes negative, the user is blocked.
5. A blocked user cannot start another session.
6. A top-up is processed to increase the wallet balance.
7. A blocked user becomes eligible again when the balance reaches zero or above.

Top-up Rules

Top-up Amount	Result
Below Rs.100	Rejected

Top-up Amount	Result
Rs.100 to below Rs.1,000	Accepted without bonus
Rs.1,000 or more	Accepted with 3% bonus

Example:

For a Rs.1,000 top-up:

Bonus = Rs.1,000 × 3% = Rs.30

Total wallet credit = Rs.1,030

Wallet Ledger

The wallet ledger stores:

- User ID
- Timestamp
- Entry type
- Reference ID
- Transaction amount
- Running wallet balance

Ledger entry types include OPENING, SESSION, TOPUP, TOPUP_BONUS, and TOPUP_REJECTED.

15. Event Processing and Timestamp Handling

The system processes events in chronological order.

When multiple events have the same timestamp, the processing priority is:

1. Top-ups
2. Session ends (unplug)
3. Session starts (plug-in)

Events with the same priority are ordered by ID.

Session Time Calculation

Connected Minutes = Charge Complete Time − Plug-in Time

Idle Minutes = Unplug Time − Charge Complete Time

The revenue from a session belongs to the charger and the date of unplugging.

All timestamps use IST and have whole-minute precision.

No wallet amount is held at plug-in. The invoice is debited at unplugging. A free faulted session records a zero-value session ledger entry.

16. Input File Design

The application reads the following input files.

16.1 chargers.csv

Column	Description
charger_id	Unique charger ID
charger_type	AC_7KW, DC_30KW, or DC_60KW
location	Charger location
city	City name

16.2 users.csv

Column	Description
user_id	Unique user ID
user_name	User name
mobile	Mobile number
tier	Membership tier
opening_balance_rs	Initial wallet balance

16.3 sessions.csv

Column	Description
session_id	Unique session ID
user_id	User ID
charger_id	Charger ID
plug_in_time	Session start time
charge_complete_time	Charging completion time
unplug_time	Session end time
kwh_delivered	Energy delivered
fault_flag	Indicates whether a fault occurred

16.4 topups.csv

Column	Description
topup_id	Unique top-up ID
user_id	User ID
timestamp	Top-up time
amount_rs	Top-up amount

16.5 commands.txt

Contains commands used to load input files, process events, and generate reports.

17. Command File Design

The application uses one command per line in `commands.txt`.

The command format is:

```
COMMAND_NAME|argument1|argument2
```

Example:

```
LOAD_CHARGERS|chargers.csv
LOAD_USERS|users.csv
LOAD_SESSIONS|sessions.csv
LOAD_TOPUPS|topups.csv
PROCESS_EVENTS
DAILY_REPORT|2026-10-12
DAILY_REPORT|2026-10-13
DAILY_REPORT|2026-10-14
DAILY_REPORT|2026-10-15
DAILY_REPORT|2026-10-16
BLOCKED_USERS
```

Blank lines and lines beginning with `#` are ignored.

18. Output File Design

The system generates four output CSV files.

18.1 invoices.csv

Contains one row for every input session, including refused sessions.

Columns:

```
session_id,user_id,charger_id,status,reason,kwh_delivered,energy_rs,time_rs,idle_rs,discount_rs,gst_rs,total_rs
```

Session statuses:

- CHARGED
- FREE_FAULT
- REFUSED

Refusal reasons:

- NONE
- LOW_BALANCE
- BLOCKED

18.2 wallet_ledger.csv

Contains wallet transactions and running balances.

Columns:

```
user_id,timestamp,entry_type,reference_id,amount_rs,balance_rs
```

18.3 daily_revenue.csv

Contains daily charger-wise revenue.

Columns:

```
date,charger_id,charger_type,sessions,kwh,revenue_rs
```

Each report date contains one row for every charger.

18.4 blocked_users.csv

Contains users whose wallet balances are negative.

Columns:

```
user_id,balance_rs
```

Output Ordering

- Invoices follow the order of sessions in `sessions.csv`.
- Wallet ledger entries follow event processing order.
- Daily revenue rows are ordered by date and charger ID.
- Blocked users are ordered by user ID.

19. Sample Data Pack

The supplied sample data pack contains:

Data	Quantity
Chargers	15
Users	60
Charging sessions	400
Wallet top-ups	172
Sample period	5 days

The sample data covers 12 October 2026 to 16 October 2026.

It includes important test scenarios such as:

- DC sessions of exactly 60 minutes and 61 minutes.
- Idle periods of exactly 10 minutes and 11 minutes.
- Idle charges exceeding the Rs.300 maximum.
- Users starting with exactly Rs.100.
- Users with insufficient wallet balances.
- Faulted sessions below 0.5 kWh.
- Faulted sessions at and above 0.5 kWh.
- Rejected top-ups.
- Top-ups with bonus amounts.
- Users becoming blocked and later becoming eligible again.

20. Testing Strategy

The project uses Python's built-in unittest framework.

The tests cover:

- Energy charge calculations.
- DC time charge calculations.
- Idle charge calculations.
- Idle charge maximum limit.
- Membership discounts.
- GST calculations.
- Minimum invoice amount.
- Faulted session conditions.
- Wallet balance checks.
- Wallet debits and credits.
- Top-up validation.

- Top-up bonus calculation.
- Blocked-user conditions.
- Wallet balance recovery.

Test Files

- `test_billing.py`
- `test_wallet.py`

Test Execution Command

```
python -m unittest -v
```

The current test suite contains 17 test cases: 10 billing tests and 7 wallet tests.

21. Assumptions and Data Conventions

1. All timestamps are in IST.
2. Timestamps use the format `YYYY-MM-DD HH:MM:SS`.
3. Timestamps have whole-minute precision.
4. Money values are represented in rupees.
5. Each money figure is rounded half up to two decimal places.
6. Top-ups are processed before session ends when timestamps are equal.
7. Session ends are processed before session starts when timestamps are equal.
8. Session refusal happens at plug-in if the user is blocked or has less than Rs.100.
9. No wallet amount is held at plug-in.
10. The invoice is debited at unplugging.
11. Revenue is assigned to the date of unplugging.
12. Free faulted sessions have a zero invoice amount.
13. A blocked user becomes eligible when the wallet balance reaches zero or above.

22. Project Deliverables

The project repository contains:

- Python source code.
- Billing and wallet modules.
- Automated test files.
- Input CSV files.
- Command file.
- Generated invoice report.
- Wallet ledger report.
- Daily revenue report.
- Blocked-users report.
- README documentation.
- Design document.

- Decision log.
- Test report.
- Demo script.

GitHub Repository

https://github.com/Deepika-P-tech/E008-ECE-025

23. Conclusion

The EV Public Charging Session Billing and Wallet Engine provides an automated backend solution for managing public EV charging operations.

It processes charging sessions, calculates invoices, applies discounts and GST, manages user wallets, processes top-ups, tracks transactions, and generates daily revenue reports.

The project demonstrates how Python programming, CSV-based data processing, business rules, and automated testing can be used to develop a structured and reliable EV charging billing system.